Module A — API — "ReClaim"

Duration: 3 hours **Stack**: Any server-side framework + MySQL

Scenario

Shanghai Pudong International Airport (PVG) has commissioned **ReClaim**, a single lost property service for the whole airport. Lost property agents register found items and process passenger claims from any terminal; passengers report lost items online and follow their claim's progress; the system suggests which stored items best match each new claim.

The staff desk application and the passenger portal are built by other teams. You deliver the REST API behind them, implemented exactly as specified below. Your implementation will be tested against an automated test suite (Bruno). Responses must match the expected structure, status codes, and business logic precisely.

Setup

- Import the provided database dump: `dist/database/reclaim-db.sql` (MySQL). The same dataset is also provided as JSON files in `dist/data/json/` as a failsafe — the SQL dump is the primary source and the assessment dataset is the MySQL database
- All endpoint paths below are **relative to your API base URL**. The base URL may sit at a domain root or under a sub-path — e.g. `http://localhost:8000/api`, `https://wsXX-YYYY-module-a/api`. Your API must work regardless of where it is mounted; you will enter your base URL in the test suite environment.
- API documentation (Swagger UI) is available in `dist/api-docs/` — open `index.html` in your browser
- All responses must be JSON — no HTML responses anywhere
- All staff endpoints require Bearer token authentication (via `POST /login`)
- All passenger portal endpoints require separate passenger Bearer token authentication (via `POST /passenger/login`)
- Public endpoints `GET /claims/track/{referenceCode}` and `POST /passenger/register` require no authentication

Testing Your Work

The same Bruno test suite used for assessment is provided in `dist/api-tests/` — use it throughout the module to check your progress.

1. Open Bruno → **Open Collection** → select the `api-tests` folder. If Bruno asks which JavaScript sandbox to use, choose **Developer Mode** — the reset step needs it
2. Select the **Local** environment and fill in:
    - `baseUrl` — wherever your API runs, e.g. `http://localhost:8000/api` or `http://localhost/<folder>/api`
    - `dbName`, `dbUser`, `dbPass` (and `dbHost` / `dbPort` if not local defaults) — your MySQL connection, so the suite can reset your data for you
    - `mysqlPath` — only if the `mysql` command is not on your PATH, set the full path to the MySQL command-line client

3. Test names carry the marking aspect (`A1.1:` … `A9.4:`) and quote its wording — a green test is the same check the assessors will run, in the same order as the marking scheme

Testing one part at a time (recommended while you build — you do not need the whole API working to test the part you are on). Right-click a folder → **Run**, in this order:

1. `A - reset` — re-imports the provided dump directly into your MySQL database. It needs none of your code, so it works from the first minute of the module (some tests change data — always start here)
2. `A1 - auth` — logs in and stores the tokens every other folder uses
3. The folder you are working on, e.g. `A3 - items`

`A8 - passenger-portal` handles its own passenger login, but still needs steps 1 and 2 first (some of its tests use the staff token).

Running everything: run from the collection root — folders execute in order, the database is reset from the dump at the start and end automatically, and the run is repeatable.

Authentication

Two separate authentication systems exist:

Staff Auth

`POST /login`

Request:

```
{ "email": "admin1@reclaim-pvg.cn", "password": "admin123" }
```

Response `200`:

```
{
  "data": {
    "token": "kJH8s2Lq...60chars",
    "user": { "id": 1, "name": "...", "email": "...", "role": "admin" }
  }
}
```

Response `401`: `{"message": "Invalid credentials"}` — also for inactive accounts.

`POST /logout` — Invalidate token. No request body. Response `200`: `{"message": "Logged out successfully"}`

All authenticated endpoints require header: `Authorization: Bearer {token}`

Unauthenticated requests return `401`: `{"message": "Unauthenticated"}`

Passenger Auth

`POST /passenger/register` and `POST /passenger/login` — see Passenger Portal section. Separate token system from staff; staff tokens are rejected on `/passenger/*` endpoints and vice versa. One deliberate exception: `GET /terminals` accepts either token (see Terminals).

Roles

- **Admin** — full access to all endpoints and all data
- **Agent** — sees only data for their assigned terminals. Can register items and process claims for their terminals. Assigned to **multiple terminals** (many-to-many); all queries scoped to their assigned terminals.

Forbidden actions return `403`: `{"message": "Forbidden"}`

Seed Data

Terminals

Code	Name
PVG-T1	Terminal 1
PVG-T2	Terminal 2
PVG-S1	Satellite Hall S1
PVG-S2	Satellite Hall S2

Staff

Email	Password	Role	Terminals
admin1@reclaim-pvg.cn	admin123	Admin	All
admin2@reclaim-pvg.cn	admin123	Admin	All
agent1@reclaim-pvg.cn	agent123	Agent	Terminal 1 (PVG-T1), Satellite Hall S1 (PVG-S1)
agent2@reclaim-pvg.cn	agent123	Agent	Terminal 2 (PVG-T2), Satellite Hall S2 (PVG-S2)

Passengers

Sample:

Email	Password	Name	Country
passenger1@email.com	passenger123	Li Wei	China
passenger2@email.com	passenger123	Maria Santos	Portugal
passenger31@email.com	passenger123	Chen Ming (deactivated account)	China

Categories

`electronics`, `documents`, `luggage`, `clothing`, `jewellery`, `keys`, `other`

Endpoints — Terminals (3 endpoints)

`GET /terminals`

All terminals with counts. No pagination. Agents see only their assigned terminals.


```
{
  "data": [
    {
      "id": 1,
      "name": "Terminal 1",
      "code": "PVG-T1",
      "description": "International departures and arrivals, gates A-D",
      "status": "open",
      "items_in_storage_count": 14,
      "open_claims_count": 9,
      "created_at": "2026-...",
      "updated_at": "2026-..."
    }
  ]
}
```


`open_claims_count` counts claims with status `submitted` or `under-review`.

`GET /terminals` also accepts a **passenger token**: passengers receive **all** terminals (this feeds the terminal selector in the passenger portal's claim form). Staff behaviour is unchanged — agents still see only their assigned terminals.

`GET /terminals/{id}`

Single terminal (same fields). Response `404` if not found. Agents get `403` for terminals not assigned to them.

`GET /terminals/{id}/items`

Found items at a terminal (paginated). Query: `?status=in-storage`

Endpoints — Found Items (5 endpoints)

`GET /items`

List found items (paginated, 15 per page, newest first). Agents see only items at their assigned terminals.

Query params: `?status=in-storage&category=electronics&terminal_id=1&search=iphone` — `search` matches `brand` or `description`.


```
{
  "data": [
    {
```

```
        "id": 1,
        "reference_code": "FI-K3M9P2QX",
        "terminal": { "id": 1, "name": "Terminal 1", "code": "PVG-T1" },
        "category": "electronics",
        "brand": "Apple",
        "colour": "black",
        "description": "iPhone 15 Pro, cracked screen protector",
        "found_on": "2026-07-01",
        "found_location": "Gate D68",
        "storage_shelf": "T1-R3-S07",
        "status": "in-storage",
        "registered_by": { "id": 3, "name": "Agent One" },
        "created_at": "2026-...",
        "updated_at": "2026-..."
      }
    ],
    "links": { "..." },
    "meta": { "current_page": 1, "last_page": 9, "per_page": 15, "total": 123 }
  }
```

`brand`, `colour`, and `storage_shelf` are nullable.

`GET /items/{id}`

Single item (same fields). Response `404` if not found. Agents get `403` if the item is not at their terminals.

`POST /items`

Register a found item. Reference code auto-generated (`FI-XXXXXXXX`). Status starts as `registered`. Agents can only register items at their assigned terminals (`403` otherwise). An activity log is auto-created.

```
  {
    "terminal_id": 1,
    "category": "electronics",
    "brand": "Apple",
    "colour": "black",
    "description": "iPhone 15 Pro, cracked screen protector",
    "found_on": "2026-07-01",
    "found_location": "Gate D68"
  }
```

`brand` and `colour` optional; `found_on` must not be in the future. Response `201` with the created item.

`PUT /items/{id}`

Update item details. All fields optional — send only what changes (partial updates are allowed):

```
  { "brand": "Sony", "colour": "navy", "storage_shelf": "T1-R2-S04" }
```

Updatable fields: `category`, `brand`, `colour`, `description`, `found_location`, `storage_shelf`. Status and reference code cannot be changed here. Response `200`: updated item.

`PATCH /items/{id}/status`

All fields are sent in the JSON request body — there are no query parameters on this endpoint.

Moving an item into storage (`storage_shelf` is required — either already set on the item, or sent together with the status):

```
{ "status": "in-storage", "storage_shelf": "T1-R3-S07" }
```

Retiring an item after the retention period:

```
{ "status": "disposed" }
```

Allowed via this endpoint:

- `registered` → `in-storage` — requires `storage_shelf` (`422` without one)
- `in-storage` → `donated` | `disposed` — **only when `found_on` is more than 60 days ago**; `422 {"message": "Item is still within the retention period"}` otherwise

`matched` and `returned` cannot be set directly (`422`) — they are managed by the claim workflow. Invalid transitions return `422`. Response `200`: updated item. Every status change writes an activity log.

Endpoints — Claims (4 endpoints)

`GET /claims`

List claims (paginated, newest first). Agents see only claims for their assigned terminals.

Query params: `?status=submitted&category=electronics&terminal_id=1`

```
{
  "data": [
    {
      "id": 1,
      "reference_code": "CL-A7B2C9DE",
      "passenger": { "id": 1, "first_name": "Li", "last_name": "Wei", "email":
"passenger1@email.com" },
      "terminal": { "id": 1, "name": "Terminal 1", "code": "PVG-T1" },
      "category": "electronics",
      "brand": "Apple",
      "colour": "black",
      "description": "Black iPhone with red case, lock screen photo of a dog",
      "lost_on": "2026-07-01",
      "flight_number": "MU583",
```

```
        "status": "under-review",
        "matched_item": null,
        "created_at": "2026-...",
        "updated_at": "2026-...",
        "resolved_at": null
      }
    ],
    "links": { "..." },
    "meta": { "current_page": 1, "last_page": 5, "per_page": 15, "total": 61 }
  }
```

`matched_item` is `null` until a match is confirmed, then: `{ "id": 12, "reference_code": "FI-...", "storage_shelf": "T1-R3-S07" }`.

`GET /claims/{id}`

Single claim (same fields). Response `404` if not found. Agents get `403` if the claim is not for their terminals.

`PATCH /claims/{id}/status`

Request: `{"status": "under-review"}`

Allowed via this endpoint only:

- `submitted` → `under-review` — an agent picks up the claim
- `matched` → `under-review` — releases the match: the linked item returns to `in-storage` and `matched_item` becomes `null`

All other statuses are managed by dedicated endpoints (`422` here). Invalid transitions return `422`. Response `200`: updated claim. Activity logs auto-created.

`GET /claims/track/{referenceCode}` (PUBLIC — no auth)

```
{
  "data": {
    "reference_code": "CL-A7B2C9DE",
    "status": "under-review",
    "category": "electronics",
    "terminal": { "name": "Terminal 1", "code": "PVG-T1" },
    "lost_on": "2026-07-01",
    "created_at": "2026-...",
    "resolved_at": null
  }
}
```

No passenger personal data is exposed. Response `404` if the reference code is not found.

Endpoints — Matching & Resolution (4 endpoints)

These four endpoints are one workflow. A typical claim travels like this:

1. A passenger files a claim → status `submitted`
2. An agent picks it up → `PATCH /claims/{id}/status` → `under-review`
3. The agent asks the system for likely items → `GET /claims/{id}/matches` (ranked suggestions)
4. The agent confirms one → `POST /claims/{id}/match` → claim **and** item both become `matched`
5. Then one of three things happens:
   - the passenger collects it → `POST /claims/{id}/resolve` → claim `resolved`, item `returned`
   - it was the wrong item → `PATCH /claims/{id}/status` back to `under-review` → link cleared, item back to `in-storage`
   - nothing suitable exists → `POST /claims/{id}/reject` (open claims only)

`GET /claims/{id}/matches`

The matching engine. Given an open claim, it scans the stored items and returns the most likely matches, best first, so an agent can see at a glance which shelf to check.

The claim must be `submitted` or `under-review` — `422` otherwise.

Step 1 — candidates. Only items with status `in-storage` and the **same category** as the claim are considered. All other items are ignored.

Step 2 — score each candidate by comparing it with the claim (maximum 100 points):

Comparison	Points
`brand` identical (case-insensitive; both values set)	30
`colour` identical (case-insensitive; both values set)	25
item is at the claim's terminal	20
days between `found_on` and `lost_on`: 0-1 days	25
days between `found_on` and `lost_on`: 2-3 days	15
days between `found_on` and `lost_on`: 4-7 days	5
days between `found_on` and `lost_on`: 8 days or more	0

Step 3 — result. Candidates scoring **less than 40 are dropped**. The rest are returned sorted by `score` (highest first); ties broken by `found_on` (newest first), then `id` (lowest first). Each entry carries the total `score` and its `breakdown`.

In pseudocode — implement exactly this, in any language:

```
candidates = items where status = "in-storage" and category = claim.category

for each item in candidates:
    breakdown.brand    = 30 if claim.brand and item.brand are both set
                            and equal ignoring case, else 0
    breakdown.colour   = 25 if claim.colour and item.colour are both set
                            and equal ignoring case, else 0
```

```
    breakdown.terminal = 20 if item.terminal_id equals claim.terminal_id, else 0

    days = absolute difference in whole days between item.found_on and
claim.lost_on
    breakdown.date     = 25 if days <= 1
                         15 if days <= 3
                          5 if days <= 7
                          0 otherwise

    score = breakdown.brand + breakdown.colour + breakdown.terminal +
breakdown.date

keep only results with score >= 40
sort by score (high to low), then found_on (new to old), then id (low to high)
return each as { item, score, breakdown }
```

Worked example — claim: jewellery, Cartier, gold, Terminal 1, lost on 1 July:

Candidate (all jewellery, in storage)	brand	colour	terminal	date	Score
Cartier gold bracelet, Terminal 1, found 1 July	30	25	20	25	**100**
Cartier gold necklace, Terminal 2, found 29 June	30	25	0	15	**70**
Tiffany silver ring, Terminal 1, found 1 July	0	0	20	25	**45**
Pandora rose-gold bracelet, Terminal 2, found 21 June	0	0	0	0	0 — not returned

Response `200`:

```
{
  "data": [
    {
      "item": {
        "id": 12,
        "reference_code": "FI-K3M9P2QX",
        "terminal": { "id": 1, "name": "Terminal 1", "code": "PVG-T1" },
        "category": "jewellery",
        "brand": "Cartier",
        "colour": "gold",
        "description": "...",
        "found_on": "2026-07-01",
        "storage_shelf": "T1-R3-S07"
      },
      "score": 100,
      "breakdown": { "brand": 30, "colour": 25, "terminal": 20, "date": 25 }
    }
  ]
}
```

An empty `data` array is returned when nothing scores 40 or more.

> **Check yourself:** the worked example above is real — in the provided seed data, claim `CL-F8DA73A7` (id 1) is exactly this jewellery claim. A correct implementation returns four suggestions scoring **100, 70, 50, 45** for it (the 50 is a brandless gold item not shown in the table). If your endpoint produces those numbers in that order, your engine is right.

`POST /claims/{id}/match`

Request: `{"item_id": 12}`

Confirm a match. Claim must be `under-review`; item must be `in-storage` (`422` otherwise — the item does not need to appear in the suggestions). Sets claim to `matched`, item to `matched`, links them. Activity logs auto-created for both.

Response `200`: updated claim with `matched_item`.

`POST /claims/{id}/resolve`

Hand the item back. No request body. Claim must be `matched` (`422` otherwise). Sets claim to `resolved` (auto-sets `resolved_at`), item to `returned`. Activity logs auto-created.

Response `200`: updated claim.

`POST /claims/{id}/reject`

Request: `{"reason": "No matching item found after 30 days"}` — `reason` optional.

Claim must be `submitted` or `under-review` (`422` otherwise; a `matched` claim must be released first). Sets claim to `rejected`. Activity log auto-created (reason in details).

Response `200`: updated claim.

Endpoints — Dashboard (3 endpoints)

`GET /dashboard/stats`

Aggregated statistics. Agents see only their assigned terminals' data.

```
{
  "data": {
    "total_items": 123,
    "items_in_storage": 67,
    "total_claims": 61,
    "open_claims": 20,
    "today_items": 7,
    "today_claims": 2,
    "items_by_category": {
      "electronics": 18,
      "documents": 23,
      "luggage": 20,
```

```
      "clothing": 21,
      "jewellery": 5,
      "keys": 14,
      "other": 22
    },
    "claims_by_status": {
      "submitted": 10,
      "under-review": 10,
      "matched": 7,
      "resolved": 20,
      "rejected": 8,
      "withdrawn": 6
    },
    "recent_claims": [
      {
        "id": 61,
        "reference_code": "CL-...",
        "passenger_name": "Li Wei",
        "category": "electronics",
        "status": "submitted",
        "date": "2026-..."
      }
    ]
  }
}
```


`open_claims` = `submitted` + `under-review`. `recent_claims` contains the **5 most recent** claims, newest first.

`GET /dashboard/activity`

20 most recent activity log entries, newest first. Agents see only logs for their terminals' items and claims. Staff actions only — every entry has an associated `user`. Passenger self-service actions (logged with no user) are not shown here.


```
{
  "data": [
    {
      "id": 1,
      "action": "claim_matched",
      "details": "Claim CL-A7B2C9DE matched to item FI-K3M9P2QX",
      "item": { "id": 12, "reference_code": "FI-K3M9P2QX" },
      "claim": { "id": 1, "reference_code": "CL-A7B2C9DE" },
      "user": { "id": 3, "name": "Agent One", "role": "agent" },
      "created_at": "2026-..."
    }
  ]
}
```


`item` or `claim` may be `null` depending on the action.

`GET /my-terminals` (agents only)

All terminals assigned to the agent, with items registered today and open claims.

```
{
  "data": [
    {
      "terminal": {
        "id": 1,
        "name": "Terminal 1",
        "code": "PVG-T1",
        "description": "...",
        "status": "open"
      },
      "todays_items": [
        {
          "id": 1,
          "reference_code": "FI-...",
          "category": "...",
          "description": "...",
          "status": "registered",
          "found_location": "..."
        }
      ],
      "open_claims": [
        {
          "id": 1,
          "reference_code": "CL-...",
          "category": "...",
          "status": "submitted",
          "passenger_name": "Li Wei"
        }
      ]
    }
  ]
}
```

Response `403` for admins.

Endpoints — Passenger Portal (9 endpoints)

Separate authentication system for passengers. All endpoints use the `/passenger/*` prefix. Passengers can only access their own data.

`POST /passenger/register` (PUBLIC — no auth)

```
{
  "first_name": "Chen",
  "last_name": "Jing",
```

```
  "email": "chen.jing@email.com",
  "phone": "+8613912345678",
  "address_1": "88 Century Avenue",
  "address_2": null,
  "city": "Shanghai",
  "postcode": "200120",
  "country": "China",
  "password": "mypassword1"
}
```

`phone` and `address_2` optional. Email must be unique (`422`); password minimum 8 characters. Response `201` — the passenger is logged in immediately:

```
{
  "data": {
    "token": "...",
    "passenger": {
      "id": 31,
      "first_name": "Chen",
      "last_name": "Jing",
      "email": "chen.jing@email.com",
      "phone": "+8613912345678",
      "address_1": "88 Century Avenue",
      "address_2": null,
      "city": "Shanghai",
      "postcode": "200120",
      "country": "China"
    }
  }
}
```

`POST /passenger/login`

Request: `{ "email": "passenger1@email.com", "password": "passenger123" }`

Response `200`: same shape as register. Response `401` for invalid credentials or inactive accounts.

`POST /passenger/logout`

No request body. Response `200`: `{"message": "Logged out successfully"}`

`GET /passenger/claims`

Own claims (paginated, newest first). Query: `?status=submitted`. Same claim structure as `GET /claims` but **without** the `passenger` object.

`GET /passenger/claims/{id}`

Single own claim. Response `404` if not found **or not owned** by the authenticated passenger.

POST /passenger/claims

File a claim. Reference code auto-generated (CL-XXXXXXXX), status submitted. An activity log is auto-created (with no user).

```
{
  "terminal_id": 1,
  "category": "electronics",
  "brand": "Apple",
  "colour": "black",
  "description": "Black iPhone with red case, lock screen photo of a dog",
  "lost_on": "2026-07-01",
  "flight_number": "MU583"
}
```

brand, colour, flight_number optional; lost_on must not be in the future. Response 201 with the created claim.

POST /passenger/claims/{id}/withdraw

Own claim only. No request body. Claim must be submitted or under-review (422 otherwise). Sets status to withdrawn. Response 200: updated claim. Activity log auto-created (no user).

GET /passenger/profile

```
{
  "data": {
    "id": 1,
    "first_name": "Li",
    "last_name": "Wei",
    "email": "passenger1@email.com",
    "phone": "+8613800138000",
    "address_1": "200 Huaihai Middle Road",
    "address_2": "Apt 12B",
    "city": "Shanghai",
    "postcode": "200021",
    "country": "China"
  }
}
```

Never exposes password or api_token.

PUT /passenger/profile

All fields optional — send only what changes:

```
{ "phone": "+8613800138099", "city": "Shanghai" }
```

Updatable fields: `first_name`, `last_name`, `email` (unique), `phone`, `address_1`, `address_2`, `city`, `postcode`, `country`, `password` (min 8 chars).

Response `200`: updated profile (same shape as `GET /passenger/profile`).

Response Structure

Success: `{"data": ...}` — single object or array **Paginated:** `{"data": [...], "links": {...}, "meta": {"current_page": 1, "last_page": ..., "per_page": 15, "total": ...}}` **Error:**

Status	Meaning	Response
401	Unauthenticated	`{"message": "Unauthenticated"}`
403	Forbidden	`{"message": "Forbidden"}`
404	Not found	`{"message": "Resource not found"}`
422	Validation failed	`{"message": "Validation failed", "errors": {...}}`
422	Business rule violated	`{"message": "<descriptive message>"}`

> **The automated test suite (Bruno) asserts the quoted messages above — and every message quoted verbatim elsewhere in this document — character for character.** Copy them exactly as written; a spelling difference fails the test. Where the table says `<descriptive message>`, the wording is your own and only the status code is asserted.

General Requirements

- All responses must be JSON
- **Error messages quoted verbatim in this document must match exactly** (for example `Unauthenticated`, `Invalid credentials`, `Item is still within the retention period`) — they are part of the specification and are asserted by the test suite. Where this document says "descriptive message" without quoting text, the wording is yours; only the status code is asserted
- **Responses must use the exact HTTP status codes specified for each endpoint** — successful `GET` requests return `200`; creations (`POST /items`, `POST /passenger/claims`, `POST /passenger/register`) return `201`; all other successful requests return `200`. Framework defaults that differ (for example NestJS returning `201` on every POST) must be overridden to match
- Paginated endpoints: 15 items per page
- Staff Bearer token authentication on all staff endpoints
- Separate passenger Bearer token authentication on all passenger portal endpoints; the two token systems are not interchangeable
- Public endpoints (`register`, claim tracking) require no authentication
- Role-based access: admin sees all, agent scoped to assigned terminals
- Passenger endpoints scoped to the authenticated passenger's own data

- Activity logs auto-generated on item registration, item status changes, claim filing, claim status changes, match, resolve, reject, withdraw
- Item and claim status transitions enforced — invalid transitions return 422
- Match suggestions computed exactly as specified — same scores, filtering, and ordering
- CORS headers enabled for cross-origin requests
- Passwords stored hashed
- **30 endpoints total**